National Real-Time Payments System

Architectural Discussion Questions

Classification: Confidential | March 2026

Table of Contents

Assumptions and Estimations

The following assumptions shape every answer in this document.

Architecture Pattern

We follow the Central Switch pattern for message routing, consistent with UPI, PIX, Faster Payments, and FedNow. The central platform is a stateless message router. Banks handle their own accounts, balances, KYC, authentication, and ACID transaction processing.

For settlement, our NRPS follows the deferred net settlement model used by UPI (10-12 daily batch cycles) and UK Faster Payments (3 daily cycles via BoE RTGS). This differs from PIX and FedNow, which both settle each transaction individually via real-time gross settlement (RTGS). Deferred net settlement is chosen for our scale (500K TPS) because RTGS at this volume would place unsustainable liquidity demands on participating banks.

Responsibility Boundaries

Owner	Responsibilities
Central Platform (our scope)	Payment switch (message routing), directory service (alias-to-bank mapping), settlement engine (net positions), fraud scoring (central ML), transaction logging, regulatory reporting, participant management, ISO 20022 API specification
Participating Banks	Customer onboarding, Aadhaar eKYC, account management, balance checks, debit/credit processing, bank-level fraud checks, user authentication (device binding, mPIN, biometric), transaction history
Payment Service Providers (PSPs)	User-facing mobile/web apps, QR code generation/scanning, payment initiation. Connect via sponsor bank, not directly to switch

Capacity Estimates

Metric	Normal	Peak (10x)	Notes
Switch messages/sec	50,000	500,000	Stateless routing, not ACID writes
Per-bank avg TPS	~1,000	~10,000	50 banks share the ACID workload
Daily transactions	160M	1.6B	40M DAU x 4 transactions
Transaction log/day	320 GB	3.2 TB	Append-only to Kafka + Cassandra
Directory size	~30 GB	N/A	150M users x 200 bytes; fits in Redis
Annual storage	~140 TB	N/A	Including logs, audit, replicas

Switch compute	Scales horizontally	N/A	Stateless; node count determined during detailed sizing

Because the switch is stateless, scaling it is straightforward; just add more pods. The real constraint sits with individual banks, each handling roughly 1,000 TPS normally and 10,000 at peak, which is manageable for any major bank.

Question 1: Systems and Subsystems

What are the various systems and subsystems required to deliver this functionality?

The platform decomposes into five layers:

Layer 1: API Gateway

Kong + NGINX. Accepts ISO 20022 messages over mTLS from participating banks. Handles rate limiting, schema validation, and request authentication.

Layer 2: Central Switch

Written in Go (recommended) for raw throughput via goroutines. Note: UPI uses Java/Spring for its switch. Entirely stateless, with no database writes in the hot path. Contains ten internal components:

- Request Receiver: Ingests ISO 20022 messages
- Message Validator: Schema and digital signature verification
- Idempotency Guard: Duplicate detection via Redis lookup
- Address Resolver: Recipient bank lookup from directory
- Transaction Router: Dispatches debit/credit instructions to banks
- Event Publisher: Emits transaction events to Kafka
- Response Assembler: Aggregates bank responses, returns status
- Circuit Breaker: Per-bank health isolation
- Timeout Manager: 30-second auto-reversal for stuck transactions
- Metrics Collector: Prometheus instrumentation

Layer 3: Business Services

- **Directory Service**: Redis Cluster (30 GB in memory, O(1) lookups) backed by PostgreSQL for durability. Maps payment aliases (phone, VPA, Aadhaar) to bank accounts.
- **Settlement Engine**: Java/Spring Boot + PostgreSQL. Consumes Kafka events, computes net positions in configurable batch cycles, generates settlement instructions for RBI.
- **Fraud Detection**: Python + TensorFlow Serving. Async scoring via Kafka in under 100ms. Flags transactions above a risk threshold (default 0.85). Complements bank-level fraud checks.
- **Participant Management**: Onboards and certifies banks/PSPs, publishes API specs and SDKs, monitors compliance.

Layer 4: Event Backbone

Apache Kafka with replication factor 3 per DC. MirrorMaker 2 for cross-DC replication. Serves as the durable event log connecting the switch to all downstream consumers: settlement, fraud, audit, and reporting.

Layer 5: Data Stores

- **Cassandra**: Append-only transaction log. Multi-DC native replication.
- **Elasticsearch**: Full-text regulatory queries and audit search at scale.
- **S3-compatible Object Storage**: Cold archival (7-year retention), KYC documents.
- **PostgreSQL**: Durable backing store for directory and settlement.

Supporting Infrastructure

- Kubernetes (3 national DCs) for container orchestration and auto-scaling
- Prometheus + Grafana + Jaeger for monitoring, alerting, and distributed tracing
- HSMs for cryptographic key management and message signing

Question 2: Information Exchange Between Systems

What kind of information is exchanged between systems? Consider frequency, speed, volume, and consistency requirements.

Flow	Pattern	Frequency	Latency	Consistency
Bank → API Gateway → Switch	Sync (mTLS/REST)	50K-500K/sec	< 50ms (switch)	Strong
Switch → Bank (debit/credit)	Sync (mTLS/REST)	50K-500K/sec	< 800ms (bank-side)	Strong
Switch → Kafka (event publish)	Async publish (acks=all)	50K-500K/sec	< 10ms	Guaranteed delivery
Kafka → Fraud Engine	Async consumer	50K-500K/sec	< 100ms scoring	Eventual
Kafka → Settlement Engine	Async batch consumer	Batch every cycle	Minutes	Eventual
Kafka → Cassandra (txn log)	Async consumer	50K-500K/sec	< 50ms	Eventual (append-only)
Kafka → Elasticsearch (audit)	Async consumer	50K-500K/sec	Seconds	Eventual
Switch → Redis (directory)	Sync lookup	50K-500K/sec	< 1ms	Read from cache
Settlement → RBI	Batch file/API	10-12 cycles/day	Minutes	Strong
Admin/Reporting → Data Stores	Sync query	Low	< 2 sec	Read-your-writes

The synchronous hot path (bank → switch → bank) handles only message routing. Everything else (logging, fraud scoring, settlement, audit) flows asynchronously through Kafka, preventing downstream slowness from contaminating transaction latency.

The switch's contribution to latency is under 50 milliseconds. The remainder of the 2-second budget is consumed by bank-side processing (balance checks, fraud checks, debit/credit).

Question 3: Major Potential Failure Points

What are the major potential failure points in this system?

Technical Failures

- Switch node crash, mitigated by statelessness. Kubernetes reschedules immediately. But if multiple nodes fail simultaneously during peak, routing capacity drops before auto-scaling catches up.
- Redis cluster failure, causing directory lookups to stop. The switch cannot resolve recipient banks, so every transaction fails.
- Kafka broker failure or partition leader election, causing momentary backpressure on event publishing. Transactions still complete (Kafka write is async), but audit logging and fraud scoring are delayed.
- PostgreSQL primary failure; settlement engine and directory backing store go read-only until Patroni promotes a standby. Settlement cycles may be delayed.
- Cross-DC network partition, the most dangerous scenario. If the active-active switch instances in different DCs cannot coordinate, duplicate transaction processing is possible.
- HSM hardware failure; message signing stops. No transactions can be validated until the backup HSM takes over.
- mTLS certificate expiry or rotation failure; all bank-to-switch communication breaks until certificates are renewed. A common operational failure in production.

Integration Failures

- Bank API timeout or error, the most frequent failure in production. A bank's systems go slow or unresponsive, and transactions to/from that bank stall or fail.
- Half-completed transactions; sender's bank debits successfully, but receiver's bank fails to credit. The money is in limbo until reversal.
- Bank-side schema drift; a participating bank deploys a change that breaks ISO 20022 compliance. Their messages start failing validation.
- Settlement file rejection by RBI; format errors or position mismatches cause a settlement cycle to be rejected.

External Dependency Failures

- RBI settlement infrastructure outage; net positions cannot be settled. Transactions continue, but counterparty risk accumulates between banks.
- Aadhaar/eKYC service outage; affects new user onboarding at banks, not existing transactions.
- DNS provider failure; global DNS routing breaks, directing traffic to wrong or dead DCs.
- Cloud/DC-level outage; an entire data centre becomes unreachable due to power, cooling, or network failure.

Question 4: Mitigation Strategies

What mitigation strategies would you design to prevent or reduce failures?

Redundancy

- Three national data centres, each running the full stack. The switch is active-active across all three. With two DCs you risk split-brain; three gives you quorum.
- PostgreSQL uses streaming replication (primary in DC-1, standbys in DC-2 and DC-3) with Patroni for automated failover. RPO within a DC is zero via synchronous replication.
- Kafka replication factor 3 within each DC, plus MirrorMaker 2 for cross-DC replication.
- Cassandra replicates natively across DCs with LOCAL_QUORUM consistency.
- Dual HSMs per data centre with automatic failover.
- Redis directory backed by PostgreSQL; if Redis cluster fails, the switch falls back to PostgreSQL reads (higher latency but functional) while Redis recovers.
- Automated certificate lifecycle management with monitoring alerts 30 days before expiry and automated rotation where possible.

Circuit Breakers and Bulkheads

- Per-bank circuit breakers on the switch. If a bank's error rate crosses a threshold over a 30-second window, the circuit opens and transactions to that bank fail immediately with a clear status code rather than waiting for timeouts.
- Bulkhead isolation: each bank connection pool is independent. A slow bank cannot exhaust resources needed by other banks. This is the single most important mitigation; one bank's problem must never become everyone's problem.

Timeout and Auto-Reversal

- 30-second hard timeout on the switch. If a transaction doesn't complete end-to-end within 30 seconds, the switch initiates an automatic reversal instruction to the debited bank.
- Idempotency guard (Redis-based) prevents duplicate processing if a bank retries a timed-out request.

Operational

- Canary deployments with automatic rollback if error rates exceed baseline.
- Chaos engineering: regular fault injection in staging environments to validate failover paths.
- 24/7 monitoring via Prometheus + Grafana with PagerDuty escalation.
- Distributed tracing (Jaeger) for rapid diagnosis of latency issues across the switch and bank adapters.

Question 5: Contingency Plans

What contingency plans are required when failures do occur?

Half-Completed Transactions

If debit succeeds but credit fails, the switch instructs the sender's bank to reverse the debit within 30 seconds. If the reversal instruction itself fails, the transaction is flagged for manual reconciliation with a 4-hour SLA. The user sees a "pending" status, not a false success.

Bank Outage

The circuit breaker opens. Users transacting with that bank get an immediate, honest error: their bank is currently unavailable. Transactions are not queued, because in real-time payments, queuing is dangerous since the sender's balance may change before the bank recovers. Users of unaffected banks see zero impact.

Deferred Settlement

If a settlement cycle fails (RBI rejection, computation error), the system switches to deferred mode. Transactions continue to process normally; users are unaffected because they already see instant confirmation. The net positions accumulate and are included in the next successful cycle. Maximum deferral window: 24 hours before regulatory escalation.

Manual Reconciliation

A dedicated reconciliation dashboard surfaces unmatched transactions by comparing the switch's Kafka/Cassandra log against bank-reported statuses. The operations team resolves mismatches with suggested corrections. This runs continuously and is the final safety net.

Communication

- Public status page updated within 5 minutes of any P0 incident.
- Bank partner notification via dedicated channels (Slack/Teams + automated email).
- Regulator (RBI) notification within 1 hour for incidents affecting > 100K users or > 10 minutes of full outage.
- PSPs receive degradation signals via the API so they can display appropriate in-app messages to users.

Question 6: Behaviour Under Normal and Peak Load

How should the system behave differently under normal daily load vs. peak national events?

Aspect	Normal (50K msg/sec)	Peak / National Event (500K msg/sec)
Switch instances	Baseline capacity	Pre-warmed scale-out (24-48 hrs before known events)
Fraud scoring	Full ML model, all features	Lightweight model: velocity + blocklist checks only
Kafka retention	72-hour hot retention	Reduced to 24 hours; older data flushed to Cassandra faster
Settlement cycles	Standard schedule (10-12/day)	Continue as normal; volume per cycle increases but logic is the same
Elasticsearch indexing	Real-time	Delayed indexing; audit data still captured in Kafka/Cassandra
Monitoring alerts	Standard thresholds	Adjusted thresholds to avoid alert storms; war room staffed
Non-critical APIs	Full access	Rate-limited (reporting dashboards, bulk queries)

Principle: protect the hot path at all costs. During peak events, shed non-essential load (reporting queries, full-feature fraud scoring, real-time Elasticsearch indexing) to ensure that the core function, routing payment messages between banks, never degrades.

Pre-warming is essential for known events. Diwali, Eid, salary days, and tax deadlines are predictable. Scale switch capacity 24-48 hours in advance based on historical patterns rather than relying on reactive auto-scaling, which introduces lag during the initial spike.

Question 7: Architectural Trade-offs

What architectural trade-offs would you make?

Consistency vs. Availability

The switch itself is stateless, so CAP theorem doesn't apply to it directly; it's just routing messages. The consistency/availability tension sits in two places:

- **Directory (Redis + PostgreSQL):** We favour availability. A slightly stale directory entry (someone changed their linked bank 2 seconds ago and the cache hasn't updated) causes a routing error that the sender retries. Acceptable. An unavailable directory halts all transactions. Unacceptable.
- **Settlement (PostgreSQL):** We favour consistency. Net positions must be mathematically correct before submission to RBI. A wrong settlement figure is worse than a delayed one.

Cost vs. Reliability

We lean toward reliability, but the Central Switch pattern keeps costs manageable by distributing the expensive ACID work to banks.

- Three DCs instead of two: ~50% more infra cost, but eliminates split-brain and gives quorum consensus.
- Open-source stack throughout (Go, Kafka, Cassandra, PostgreSQL, Redis, Prometheus): avoids per-node licensing. At national scale, commercial observability tools alone can exceed $1M+/year.
- Tiered storage (hot/warm/cold): reduces the 7-year regulatory retention cost by approximately 60%.

Real-Time vs. Eventual Processing

Clear split:

- **Real-time (synchronous):** Message routing, directory lookup, idempotency check. These are in the hot path and must complete within 50ms on the switch side.
- **Eventual (asynchronous):** Fraud scoring, settlement computation, audit indexing, regulatory reporting. All flow through Kafka and tolerate seconds to hours of delay.

The Central Switch pattern makes this split natural. The switch does the minimum synchronous work and pushes everything else to the event backbone.

Security Depth vs. User Experience

Authentication is not the switch's responsibility; it is enforced at the bank/PSP layer. But the architecture mandates a four-layer model:

- Layer 1: Device binding (invisible to user, no friction)

- Layer 2: mPIN on every transaction (low friction, mandatory)
- Layer 3: Biometric (optional, bank/PSP risk policy)
- Layer 4: OTP for high-value transactions (higher friction, justified by risk)

The trade-off is calibrated by risk. Low-value payments have minimal friction. High-value or anomalous patterns trigger additional verification. This is the same model UPI uses and it has proven effective at balancing security and adoption.

Question 8: Global Comparisons and Recommended USPs

Refer to similar global implementations and highlight the USPs and additional features that your solution recommends.

Global Benchmark Comparison

System	Country	Scale	Key Lesson for NRPS
UPI	India	~20B txns/month, 500M+ users	Central Switch + deferred net settlement (10-12 daily cycles). Proven at massive scale. PSP ecosystem drives adoption.
PIX	Brazil	Billions of txns/month, 175M users	Central Switch + RTGS (not deferred net). Central bank operated. ISO 20022 native. DICT-based alias resolution.
Faster Payments	UK	Hundreds of millions/month, ~53M adults	Central Switch + deferred net settlement (3 daily cycles via BoE). Longest-running system (since 2008). Migrating from ISO 8583 to 20022.
FedNow	USA	Growing, 1,300+ FIs	Central Switch + RTGS. ISO 20022 from day one. Designed for interoperability with legacy (ACH, Fedwire).

Recommended USPs

1. ISO 20022 Mandatory from Day One

UPI uses a proprietary XML-based message format rather than ISO 20022. Faster Payments is mid-migration to ISO 20022 as part of the UK's New Payments Architecture (NPA). Both demonstrate the cost of not starting native. In contrast, PIX and FedNow adopted ISO 20022 from day one, which is the approach we recommend for NRPS.

Impact: Saves 2-3 years of future migration. Enables richer remittance data that improves fraud detection and reconciliation.

2. Async Fraud Scoring (Not Inline Blocking)

Most systems either skip central fraud checks (leaving it entirely to banks) or run blocking inline checks that add latency. Our approach, async scoring via Kafka with configurable thresholds, adds zero latency to the hot path while still catching high-risk patterns centrally.

Impact: Central fraud visibility across all banks without any transaction latency cost. Flagged transactions are handled post-facto, not blocked inline.

3. Configurable Settlement Cycles

Rather than hard-coding settlement frequency, the engine supports configurable cycles. Banks can be settled more frequently during high-volume periods to reduce counterparty risk, and less frequently overnight.

Impact: Reduced counterparty exposure during peak events. Flexible enough to adapt to regulatory changes without code changes.

4. Cross-Border Ready Architecture

The ISO 20022 message standard, the directory service's alias resolution model, and the settlement engine's net position calculation are all designed to extend to multi-currency and cross-border scenarios. Adding an international partner means onboarding them as a "virtual bank" in the directory and adding currency conversion to the settlement engine.

Impact: International expansion becomes a configuration and partner onboarding exercise (6-9 months) rather than a re-architecture (18-24 months).

5. Open API Platform for PSP Ecosystem

UPI's biggest success factor was not the technology; it was the third-party PSP ecosystem (PhonePe, Google Pay, Paytm). NRPS publishes comprehensive API specs, SDKs, sandbox environments, and a certification programme to enable PSPs to build on the rails.

Impact: Ecosystem-driven growth. Every PSP building on NRPS increases transaction volume without additional central platform investment.

ROI Summary

Feature	Investment	Expected Return	Payback
ISO 20022 Native	15-20% higher initial build	Avoids 2-3 year migration; cross-border ready	18-24 months
Async Fraud Scoring	Kafka + ML infra already in stack	Zero latency cost; central fraud visibility	Immediate
Configurable Settlement	Minimal (parameterization)	Reduced counterparty risk; regulatory flexibility	Immediate
Cross-Border Ready	10-15% architecture overhead	6-9 months intl launch vs 18-24 months	On first expansion
PSP API Platform	3-4 months build + DevRel team	Ecosystem-driven volume growth	6-12 months

Technology Selection: Why These Choices

Below are the key technology choices, the alternatives evaluated, and why they were rejected.

Component	Selected	Rejected Alternatives	Rationale
Switch Language	Go (recommended)	Java, Erlang/Elixir, Node.js	Go's goroutine model handles massive concurrency natively with a simple deployment model. Note: UPI uses Java/Spring for its switch; Java's thread-per-request model needs reactive frameworks for this throughput. Erlang/Elixir excels at message passing (WhatsApp, RabbitMQ) but has a significantly smaller talent pool in India. Node.js is single-threaded and requires clustering for CPU-bound validation work.
Message Broker	Apache Kafka	RabbitMQ, Pulsar, NATS	Log-based (replayable), proven at 500K+ msg/sec (LinkedIn, Uber). RabbitMQ doesn't scale to this throughput. Pulsar is viable but smaller ecosystem. NATS lacks durable replay.
Transaction Log	Cassandra	MongoDB, DynamoDB, CockroachDB	Append-only write pattern, multi-DC native replication, proven at massive scale. MongoDB is a general-purpose document store, not optimized for high-throughput append-only workloads. DynamoDB creates AWS lock-in. CockroachDB adds SQL overhead unnecessary for append-only writes.
Directory Cache	Redis Cluster	Memcached, Hazelcast	Sub-millisecond O(1) lookups, data structures for idempotency (TTL keys). Memcached lacks persistence. Hazelcast adds JVM dependency.
Settlement DB	PostgreSQL + Patroni	Oracle, MS SQL Server, MySQL, CockroachDB	ACID for batch settlement, rich SQL feature set for financial calculations, Patroni for HA, zero licensing cost. Oracle and MS SQL Server are both proven in banking but introduce vendor lock-in and significant licensing cost, which conflicts with the open-source mandate. CockroachDB is unnecessary for a single-primary batch workload.

Fraud Scoring	TensorFlow Serving	ONNX Runtime, SageMaker	Mature model serving, async via Kafka so latency is not critical. ONNX is viable but TF has a larger model ecosystem. SageMaker creates AWS lock-in.
Orchestration	Kubernetes (3 DCs)	Docker Swarm, Nomad	Industry standard. Nomad is simpler but smaller ecosystem for banking/fintech tooling. Swarm is no longer actively developed as a primary orchestration solution.
Observability	Prometheus + Grafana + Jaeger	Datadog, New Relic, Splunk	Open-source. At national scale, commercial monitoring costs escalate significantly with per-host licensing. Self-hosted stack costs a fraction.

Architecture Decision Records

ADR-001: Central Switch with Federated Banks

Context: National payment systems need a scalable, interoperable architecture. A monolithic approach would centralise all transaction processing and create a bottleneck.

Decision: Adopt the Central Switch pattern. The switch is a stateless message router. Banks handle their own accounts, balances, KYC, and ACID transaction processing.

Trade-offs: The switch cannot enforce balance checks directly. Misbehaving banks are detected through reconciliation and regulatory enforcement, not prevented in real-time.

ADR-002: Deferred Net Settlement (Batch Cycles)

Context: Users expect instant confirmation, but real-time gross settlement for every transaction at 500K TPS is impractical and unnecessary.

Decision: Instant user confirmation via the switch's transaction log. Inter-bank settlement in configurable batch cycles (comparable to UPI's 10-12 daily cycles).

Trade-offs: Banks carry counterparty risk between transaction and settlement. This model is standard and proven by UPI and UK Faster Payments. PIX and FedNow avoid this risk by using RTGS, but at the cost of higher per-transaction liquidity requirements.

ADR-003: Four-Layer Authentication at Bank/PSP Layer

Context: Payment security requires strong authentication without impacting transaction speed at the switch level.

Decision: Device binding + mPIN (mandatory) + biometric (optional) + OTP (high-value). Enforced by banks and PSPs, not the central switch. Aligned with RBI guidelines.

Trade-offs: Requires all participants to implement the authentication framework consistently. Enforced via certification programme.

ADR-004: Go for Switch, Java for Business Services

Context: The switch requires raw throughput (500K msg/sec). Business services (settlement, fraud) require rich banking integrations and batch processing.

Decision: Go for the stateless switch (goroutine concurrency, sub-ms routing). Java/Spring Boot for settlement, fraud orchestration, and reporting. Note: UPI uses Java/Spring for its switch; Go is our recommendation for raw routing throughput.

Trade-offs: Two-language codebase. Mitigated by clear service boundaries; the switch team and business services team are separate anyway.

ADR-005: Kafka + Cassandra for Transaction Log

Context: The transaction log is append-only, write-heavy, and requires multi-DC replication. No SQL queries needed in the hot path.

Decision: Kafka for event streaming (replayable, ordered). Cassandra for durable multi-DC storage. Elasticsearch for queryable audit.

Trade-offs: Cassandra's eventual consistency is acceptable for an append-only log where records are never updated. The trade-off is justified by write throughput and native multi-DC support.

ADR-006: ISO 20022 Mandatory Message Standard

Context: Interoperability between diverse bank systems requires standardisation. ISO 20022 is adopted globally by SWIFT, SEPA, and the UK's NPA.

Decision: Mandate ISO 20022 for all switch communication. Publish specifications, SDKs, and a certification programme for participants.

Trade-offs: Banks must build or adapt ISO 20022 adapters, which adds onboarding effort. However, this is now a standard industry expectation and positions NRPS for cross-border expansion.